

Faculty of Information Technology (FIT) – Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 – Software Testing (AI-augmented · 2026)

1. Student information

- **Student name (printed):** Trần Trí Nhân
- **Student ID:** 23127097
- **Class / Cohort:** Kiểm thử phần mềm - 23KTPM1
- **Assignment ID:** HW06
- **Assignment date:** 16/08/2026
- **AI tool(s) used:** Claude
- **AI tool(s) used:**
 - ☒ Yes
 - ☐ No

2. Audit Table

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
Tool: Claude Time: 12:58 15/08/2026 Input files: api_specification.md, README.md Prompt: "Your job is generating 35 Postman test cases for a single API endpoint: POST /api/register. The cases must cover: domain partitions on every input parameter(if possible), state transitions, security, and schema validation(identify parameters → establish partitions → boundaries → states → security requirements → schema constraints). First read the README and api_specification documents fully, not just skimming, to understand the requirements. Give me direct list of test cases, format as: ID, Objective, Input(if needed), Expected result, What is covered. API always takes	pool A generated test cases	INCOMPLETE	1 test case was INVALID because it invented whitespace trimming requirement(no mention in README or API specification). 1 test case was INCOMPLETE because its expected output was ambiguous.	Changed expected outcome of the INVALID test case to 400 Bad Request. Changed expected outcome of INCOMPLETE test case to 200 OK.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
higher precedence, that is if the API spec says this API does not have a field, then it doesn't, do not force because of README. DO NOT attempt to create Postman collection, DO NOT invent functional or security requirements, double check if it's in README. Avoid ambiguous expected outcome, assertions must be deterministic, no "either/or", no multiple status codes. Avoid making the test cases depend on another API endpoint to validate. Please be aware that some security requirements in README might be UI-related and irrelevant for API testing. For the injection-related test cases, please be aware that this is API-level which return JSON responses, not UI-level. Example of a test case: TC01 — Objective: This is an objective Input: {...} Expected: 200 OK; response contains ... Covers: ..."				
Tool: Claude Time: 13:32 15/08/2026 Input files: api_specification.md, README.md Prompt: "Your job is generating 35 Postman test cases for a single API endpoint: GET /api/cart. The cases must cover: domain partitions on every input parameter(if possible), state transitions, security, and schema validation(identify parameters → establish partitions → boundaries → states → security requirements → schema constraints). First read the README and api_specification documents fully, not just skimming, to understand the	pool B generated test cases	INCOMPLETE	The generated test suite contained 3 INVALID test cases.	Changed expected outcome of TC11 and TC34 to 200 OK. Removed TC21 completely.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
requirements. Give me direct list of test cases, format as: ID, Objective, Input(if needed), Expected result, What is covered. API always takes higher precedence, that is if the API spec says this API does not have a field, then it doesn't, do not force because of README. DO NOT attempt to create Postman collection, DO NOT invent functional or security requirements, double check if it's in README. Avoid ambiguous expected outcome, assertions must be deterministic, no "either/or", no multiple status codes. Avoid making the test cases depend on another API endpoint to validate. Please be aware that some security requirements in README might be UI-related and irrelevant for API testing. For the injection-related test cases, please be aware that this is API-level which return JSON responses, not UI-level. Example of a test case: TC01 — Objective: This is an objective Input: {...} Expected: 200 OK; response contains ... Covers: ..."				
Tool: Claude Time: 13:45 15/08/2026 Input files: api_specification.md, README.md Prompt: "Your job is generating 35 Postman test cases for a single API endpoint: POST /api/categories. The cases must cover: domain partitions on every input parameter(if possible), state transitions, security, and schema validation(identify parameters → establish partitions → boundaries → states → security	pool C generated test cases	INCOMPLETE	1 test case was INCOMPLETE because expected result was ambiguous.	Changed expected outcome of that test case to except only 1 final value.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
	requirements → schema constraints). First read the README and api_specification documents fully, not just skimming, to understand the requirements. Give me direct list of test cases, format as: ID, Objective, Input(if needed), Expected result, What is covered. API always takes higher precedence, that is if the API spec says this API does not have a field, then it doesn't, do not force because of README. DO NOT attempt to create Postman collection, DO NOT invent functional or security requirements, double check if it's in README. Avoid ambiguous expected outcome, assertions must be deterministic, no "either/or", no multiple status codes. Avoid making the test cases depend on another API endpoint to validate. Please be aware that some security requirements in README might be UI-related and irrelevant for API testing. For the injection-related test cases, please be aware that this is API-level which return JSON responses, not UI-level. Example of a test case: TC01 — Objective: This is an objective Input: {...} Expected: 200 OK; response contains ... Covers: ..."			

3. Summary of AI Accuracy

Metric	Count	Percentage
Total AI-generated artifacts audited	3	
VALID (correct, accepted as-is)	0	0%
INVALID (wrong; rejected)	0	0%

Metric	Count	Percentage
INCOMPLETE (acceptable after edits)	3	100%

4. Conclusion

In conclusion, Claude did really well when generating API test cases in this homework. For each API, the test suite correctly covered domain partitions, state transitions, security, and schema validation. Most of the generated test cases were VALID with well-defined payloads and deterministic expected results. The AI also discovered some interesting edge test cases for the APIs. However, because the AI had to create a lot of test cases for each feature, not all of the test cases had the same level of quality and one of them was even completely out of scope. Most of the INVALID or INCOMPLETE test cases were caused by the AI providing ambiguous expected outcome or inventing undocumented requirement. For future usages, I will explicitly note these mistakes for the AI before asking it to give the list of API test cases.

5. Mandatory Disclosure

The test suites were initially generated by Claude; I reviewed and modified TC14 and TC28 in pool A suite, TC11, TC21, TC34 in pool B suite, TC31 in pool C suite; the main report, the bug report and the CI/CD report were written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category.

Signature

- **Student name (printed):** Trần Trí Nhân
- **Student ID:** 23127097
- **Class / Cohort:** Kiểm thử phần mềm - 23KTPM1
- **Course:** CS423 / CSC13003 – Software Testing
- **Instructor:** Lâm Quang Vũ, Hồ Tuấn Thanh, Trương Phước Lộc
- **Date:** 16/08/2026
- **Signature:**



Reference

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.